Graph RAG

Mohammad Hosein Rabiee Amir Hosein Karimzadgan

Department of Computer Science and Statistics
Faculty of Mathematics
K. N. Toosi University of Technology

Overview

1. **Introduction to Graph RAG:** motivation, main use cases, and core methods
2. **Use Case I:** Enhancing multi-hop question answering performance
3. **Use Case II:** Global summarization of large document collections
4. **The Gap:** Current GraphRAG requires exposing raw sensitive data to external LLMs

Defining Retrieval-Augmented Generation (RAG)

- **What is RAG?** RAG is a technique that supplements Large Language Models (LLMs) with external knowledge to enhance factual accuracy and trustworthiness.
- **The Core Process:** It typically involves two major steps:
  - **Retrieval:** Given a user question, the system uses an index to find the most relevant chunks of information from a large corpus.
  - **Generation:** The LLM uses these retrieved chunks as context to generate an informed response.
- **The Problem it Solves:** RAG mitigates **hallucinations**, which occur when LLMs generate incorrect outputs due to missing domain-specific, real-time, or proprietary knowledge.

Why We Use Graph-Based RAG

- **Limitation of Vanilla RAG:** Standard RAG retrieves knowledge directly from isolated text chunks, which may fail to capture rich semantic links between entities.
- **The Graph Advantage:** Graph-based RAG employs graph structures (nodes and edges) built from the corpus to enhance contextual understanding.
- **Key Benefits:**
  - **Captures Relationships:** Explicitly models links between entities (e.g., Real Sociedad winning Copa Del Rey in a specific year).
  - **Improved Accuracy:** Enhances factual accuracy, adaptability, and interpretability.
  - **Structured Representation:** Edges can be enriched with contextual properties such as temporal data and attributes.

Vanilla RAG vs. Graph-based RAG

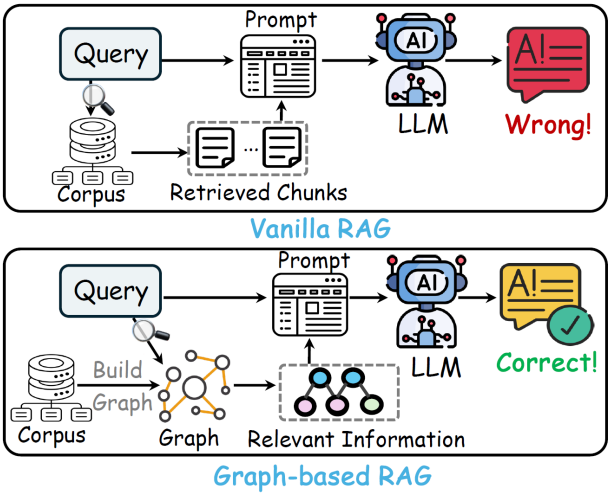


Figure: Overview of vanilla RAG and graph-based RAG.

Introducing the Unified Framework

- **Overview:** To systematically compare different methods, the sources propose a unified framework consisting of four distinct stages [Zhou et al., 2025]:

  1. **Graph Building:** Transforming the input corpus into a graph (e.g., Knowledge Graph, Tree, or Passage Graph) by extracting nodes and edges.
  2. **Index Construction:** Storing graph elements (nodes, relationships, or communities) in a vector database or computing community reports.
  3. **Operator Configuration:** Selecting retrieval operators (e.g., Vector Search, Personalized PageRank).
  4. **Retrieval & Generation:** Converting the user question into a searchable unit and generating the final answer.

Unified Framework of Graph-based RAG

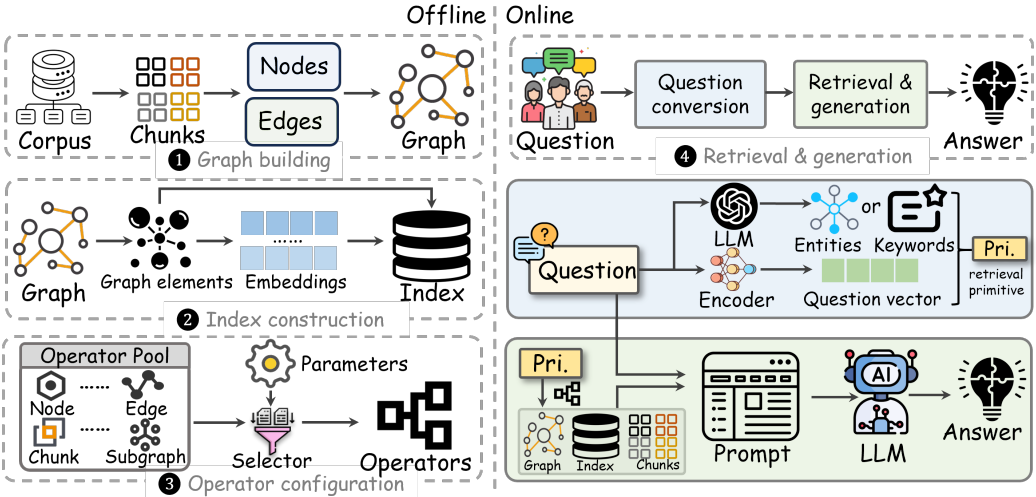


Figure: Workflow of graph-based RAG methods under a unified framework.

Challenges in Multi-Hop Question Answering

Definition

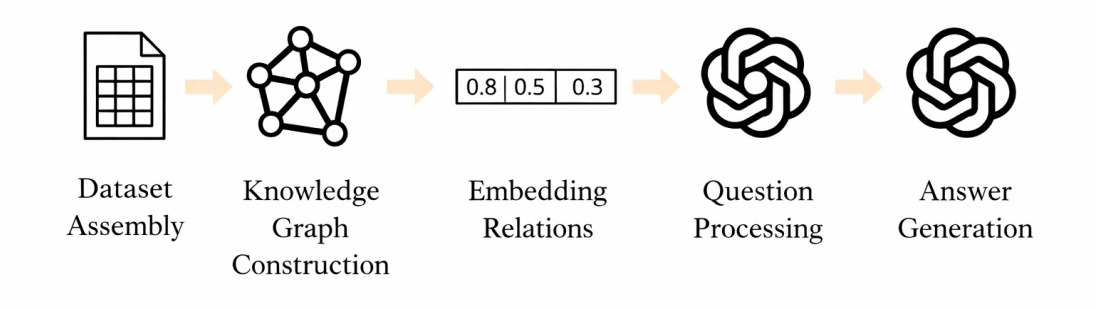
Multi-hop QA requires reasoning over multiple pieces of information or documents to answer complex queries.

- **Core Difficulties:**
  - Necessitates multi-step reasoning.
  - Requires integration of information from disparate sources.
- **Shortcomings of Existing Methods:** Traditional approaches often struggle with effective retrieval across documents, leading to incomplete or inaccurate answers.
- **The MuSiQue Dataset:** A benchmark designed for 2–4 hop questions to improve performance on genuine multihop tasks.

Research Gap and Proposed Solution

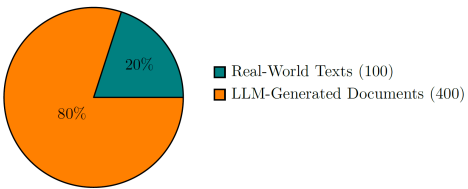
- **The Problem:** Traditional RAG models often fail to integrate information from disparate sources into coherent and accurate answers for multi-hop tasks.
- **Proposed Innovation:** A novel approach leveraging vector embedding retrieval from a graph relations database. [Amiri Shavaki et al., 2024]
- **Mechanism:** By constructing a knowledge graph and embedding its relations, the method explicitly models relationships between entities and concepts to enhance multi-hop reasoning.

Overall Pipeline



Dataset Assembly and Composition

- **Scale:** The dataset consists of 500 short textual documents and 296 multi-hop questions.
- **Multi-Hop Requirement:** Questions are specifically designed so that no single document contains the full answer; retrieval of at least two documents is required.
- **Data Sources:**
  - 20% (100 documents): Sourced from real-world internet texts.
  - 80% (400 documents): Generated by an LLM and validated by human supervisors.
- **Purpose:** This curated dataset serves as a benchmark to compare the proposed Graph RAG method against traditional RAG approaches.



Composition of the dataset.

Knowledge Graph Construction

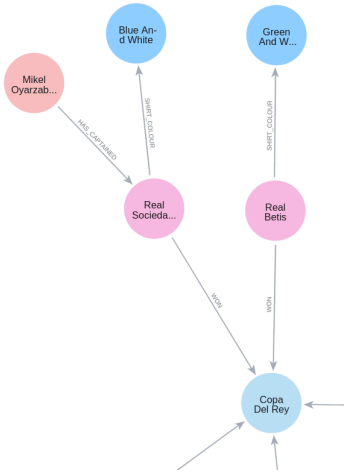
- **Extraction Process:** An LLM extracts entities (represented as nodes) and relationships (represented as edges) from the documents.
- **Merging Methodology:**
  - Documents are processed individually to extract relations.
  - Individual graphs are then merged into a unified, comprehensive graph.
- **Efficiency Finding:** Research indicated that merging separate graphs captures relations more accurately than attempting a single large-batch LLM extraction, which risks missing critical connections.
- **Contextual Properties:** Beyond simple nodes and edges, the graph stores temporal data (dates/times) and attributes as properties.
- **Management:** Neo4j is used as the graph database to manage these complex, interconnected data structures.

Enriched Semantic Representation

Example

- **Relation 1:** (Real Sociedad, WON, Copa Del Rey) with property year = 2020.
- **Relation 2:** (Real Sociedad, SHIRT_COLOUR, Blue And White).

- **Reasoning Capability:** This structure allows the system to answer complex queries like *"What color did the 2020 Copa del Rey winners wear?"* by navigating through specific properties.



Sample subgraph of the constructed knowledge graph.

Embedding Relations in a Vector Database

- **RAG-based Retrieval**

  For the retrieval step, relations in the knowledge graph are embedded into a vector database to enable fast similarity search.

- **Structured Embedding Construction**

  To create the embeddings, we use:

  - the names of the two head entities,
  - the relation itself,
  - and all associated properties (including dates and times).

- **Context Retention**

  Including temporal information ensures that important context is preserved during retrieval.

- **Vector Storage and Similarity Search**
  - FAISS is used for efficient indexing based on approximate search algorithms.
  - Retrieval is performed using Euclidean distance (L2 norm) to identify semantically similar relations.

Answer Generation Workflow

- **Step 1: Identifying Incomplete Relations**

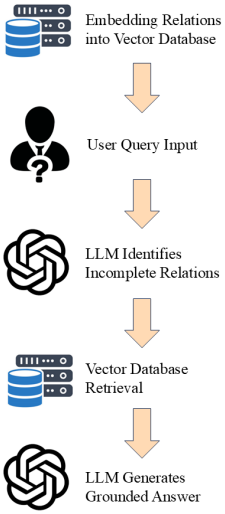
  The user query is passed to an LLM, which identifies missing or incomplete relations required to answer the question.

- **Step 2: Grounded Retrieval**

  The vector database retrieves the most relevant relation embeddings to complete the missing connections using similarity search.

- **Step 3: Grounded Answer Synthesis**

  A second LLM call generates the final answer, explicitly restricted to the retrieved relations from the knowledge graph. This grounding strategy prevents hallucination and ensures answers are derived from the knowledge graph rather than the LLM's internal knowledge.



Experimental Settings

Table I: Configuration of the Proposed Method

Component	Configuration
Graph Database	Neo4j
Vector Database	FAISS
Vector Similarity Metric	Euclidean Distance (L2)
Embeddings	text-embedding-ada-002
Knowledge Graph Construction	GPT-4o
Relation Extraction	GPT-4o
Answer Generation Model	GPT-3.5-turbo

Table II: Configuration of the Baseline RAG Approach

Component	Configuration
Vector Database	Weaviate
Vector Similarity Metric	Euclidean Distance (L2)
Embeddings	text-embedding-ada-002
Answer Generation Model	GPT-3.5-turbo

Evaluation Metrics: RAGAS Framework

- **RAGAS Framework**
  - **Answer Similarity**: semantic similarity between generated answer and ground truth, computed using cosine similarity of embeddings
  - **Answer Correctness**: weighted combination of factual correctness and semantic similarity

Factual Correctness (F1 Score)

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Answer Correctness Score

$$\text{Answer Correctness} = w_f \times F1 + w_s \times \text{Semantic Similarity}$$

$$w_f = 0.75 \quad , \quad w_s = 0.25$$

Evaluation Metrics: LLM Evaluator

- **LLM Evaluator**: head-to-head comparison without predefined ground truth
- **Methodology**:
  - LLM receives the question and answers from both methods
  - LLM selects the superior answer
  - repeated for all questions in the dataset
- **Qualitative Dimensions**:
  - **Comprehensiveness**: depth and completeness
  - **Empowerment**: usefulness and actionability
  - **Directness**: clarity and interpretability
  - **Diversity**: variation in responses
- **Outcome**: aggregated win/loss counts comparing proposed method and baseline

RAGAS Results

Table III: RAGAS Framework Results on Both Datasets

Metric	Proposed	Baseline	Proposed	Baseline
	Our Dataset		MuSiQue Testset	
Correctness	0.649	0.493	0.477	0.382
Similarity	0.948	0.905	0.928	0.876

Table IV: RAGAS Metric Comparison Counts

Metric	Proposed	Baseline	Proposed	Baseline
	Our Dataset		MuSiQue Testset	
Correctness	186	110	17	8
Similarity	170	126	18	7

LLM Evaluator Results

Table V: LLM Evaluator Results on Both Datasets

Dimension	P	B	P	B
	Our Dataset		MuSiQue Testset	
Comprehensiveness	224	72	20	5
Diversity	155	141	14	11
Empowerment	246	50	21	4
Directness	162	134	16	9

P: proposed method wins B: baseline method wins

References

Amiri Shavaki, M., Omrani, P., Toosi, R., and Akhaee, M. A. (2024).
Knowledge graph-based retrieval-augmented generation for multi-hop avir.
IEEE Access, 12:XXX–XXX.

Zhou, Y., Su, Y., Sun, Y., Wang, S., Wang, T., He, R., Zhang, Y., Liang, S., Liu, X., Ma, Y., and Fang, Y. (2025).
In-depth analysis of graph-based rag in a unified framework.
Proceedings of the VLDB Endowment, 18(1).
arXiv:2503.04338.

From Local to Global: A GraphRAG Approach to Query-Focused Summarization

- **Authors:** Darren Edge, Ha Trinh, et al. (Microsoft Research)
- **Problem:** Conventional RAG performs well for fact-level queries but struggles with *global questions* that require understanding an entire corpus
  - Example: "What are the main themes in this dataset?"
- **Objective:** Design a scalable QA framework over private text corpora that handles
  - **Query generality:** from specific facts to high-level themes
  - **Data scale:** collections with 1M+ tokens
- **Core Innovation — GraphRAG:** A graph-based index built using LLMs
  - **Entity Knowledge Graph:** entities and relations extracted from documents
  - **Community Summaries:** pre-computed summaries of related entity groups

The Gap in Conventional Vector RAG

- **Mechanism:** Standard vector RAG retrieves documents based on semantic similarity in embedding space.
- **The Sensemaking Problem:** Global tasks require understanding interconnected entities, events, and themes across an entire corpus.
- **Failure Mode:** Vector RAG is optimized for fact retrieval, not query-focused summarization over large document collections.

The GraphRAG Pipeline Overview

- **Indexing Phase:**
  - Source Documents → Text Chunks
  - Entity & Relationship Extraction
  - Knowledge Graph Construction
  - Community Detection
  - Community Summarization
- **Query Phase:**
  - User Query → Community Answers (Map)
  - Aggregation into Global Answer (Reduce)
- **Core Strength:** Leverages graph modularity for parallel, hierarchical summarization.

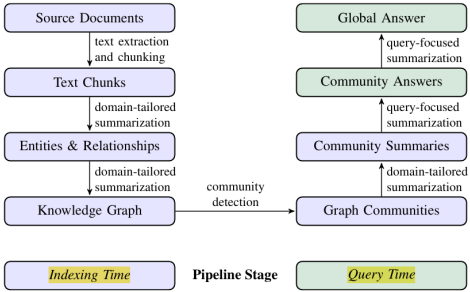


Figure 1: High-level GraphRAG indexing and query pipeline.

Entity Extraction and the “Gleaning” Technique

- **Chunking Strategy:** Documents are split into 600-token chunks to maximize information recall. (larger chunks reduce cost but may degrade information recall)
- **Extraction:** LLM extracts entities (name, type, description) and relationships (description, strength).
- **Self-Reflection (Gleaning):**
  - LLM evaluates whether entities were missed.
  - Additional extraction passes recover missing elements.

From Unstructured Text to a Knowledge Graph

- **Abstractive Summarization:** Entity and relationship extraction acts as a conceptual summary of the text.
- **Aggregation:** Repeated entities and relationships across documents are merged into unified nodes and edges.
- **Resilience:** Despite simple string matching, duplicate entities naturally cluster during community detection.

Partitioning the Graph with the Leiden Algorithm

- **Modular Partitioning:** Strongly connected nodes are grouped into communities.
- **Leiden Algorithm:** Applied recursively to generate a hierarchy of communities.
- **Scalability:** Enables divide-and-conquer summarization from root-level (global) to leaf-level (local) communities.

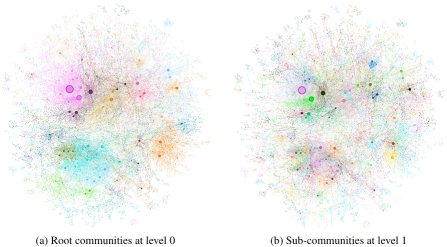


Figure 4: Hierarchical community structure induced by the Leiden algorithm

Generating Hierarchical Community Reports

- **Report Structure:**
  - Executive summary
  - Impact rating
  - Detailed findings grounded in graph data
- **Bottom-Up Generation:** Leaf-level summaries emphasize entity and relationship details.
- **Context Management:** Higher-level summaries replace detailed content with concise sub-community summaries.

Executing Global Queries via Map-Reduce

- **Preparation:** Community summaries are shuffled and chunked to balance context distribution.
- **Map Step:** Parallel generation of intermediate answers with helpfulness scores (0–100).
- **Reduce Step:** High-scoring answers are filtered, ranked, and merged to produce the final response.

Benchmarking Global Sensemaking

- **Motivation:** Existing RAG benchmarks focus on factoid retrieval.
- **Adaptive Evaluation:**
  - Generate hypothetical personas
  - Identify persona-specific tasks
  - Create global, corpus-level questions
- **Scale:** 125 questions generated per dataset (Podcasts and News).

Defining Evaluation Criteria

- **Comprehensiveness:** Coverage of all relevant aspects of the question.
- **Diversity:** Variety of insights, viewpoints, and themes.
- **Empowerment:** Ability to support informed reasoning with evidence.
- **Directness (Control):** Conciseness and specificity used to validate other metrics.

Performance Comparison: GraphRAG vs. Vector RAG

- **Dominance:** GraphRAG outperforms vector RAG in
  - Comprehensiveness (72–83% win rate)
  - Diversity (62–82% win rate)
- **Trade-Off:** Vector RAG excels in Directness but lacks global context.
- **Consistency:** Results hold across Podcast and News datasets.

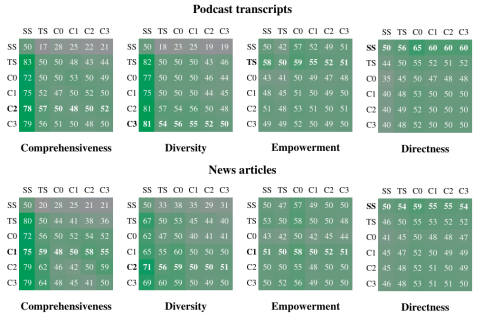


Figure 2: Win-rate comparison across evaluation metrics

Balancing Performance and Token Cost

- **Root-Level Efficiency (C0):**
  - Requires 9x–43x fewer tokens than full text summarization
  - Still outperforms vector RAG
- **Hierarchical Scaling:** Lower-level summaries (C3) use 26–33% fewer tokens with best performance.
- **Conclusion:** C0 summaries provide an efficient entry point for iterative sensemaking.

Validating Results with Claim-Based Measures

- **Factual Claims:** Verifiable statements extracted using Claimify[1].
- **Validation Results:**
  - Higher number of unique claims (Comprehensiveness)
  - More claim clusters (Diversity)
- **Alignment:** 78% agreement between claim-based metrics and LLM-as-a-judge scores.

[1] Claimify (Metropolitansky & Larson, 2025) is an LLM-based method that identifies sentences containing at least one factual claim and decomposes them into simple, self-contained, verifiable claims.

Key Takeaways

- Vector RAG fails on global sensemaking tasks.
- GraphRAG introduces a scalable, graph-based summarization framework.
- Hierarchical community summaries enable efficient, global reasoning over 1M+ tokens.
- GraphRAG significantly improves comprehensiveness and diversity while controlling token cost.

SoK: Privacy Risks and Mitigations in Retrieval-Augmented Generation Systems

- **Authors:** Andreea-Elena Bodea, Stephen Meisenbacher, Alexandra Klymenko, Florian Matthes (Technical University of Munich)
- **Goal:** First systematization of privacy risks, mitigations, and evaluation strategies in RAG systems via a systematic literature review (72 papers).
- **Key Artifacts:**
  - Taxonomy of RAG privacy risks
  - RAG privacy process diagram

The Two-Sided Coin: Leakage vs. Adversarial Manipulation

- **Threat Landscape:** Privacy risks fall into:
  - **Leakage:** Inherent system vulnerabilities
  - **Adversarial Manipulation:** Malicious attacks
- **Conceptual Difference:**
  - Leakage focuses on *what* is exposed
  - Attacks focus on *how* exposure is triggered
- **Leakage Spectrum:** Sensitive data may leak at any pipeline checkpoint (chunking, encoding, retrieval, generation).

Risk Point 1: Dataset Leakage

- **Technical Risks:**
  - Insecure storage (e.g., unprotected cloud drives)
  - Weak internal access controls
- **PII Exposure:** Non-malicious leakage when sensitive data is indexed without masking or sanitization.
- **Mitigations:**
  - Automated PII detection and redaction
  - Strong access control policies
  - Synthetic data generation
  - Text rewriting to preserve semantics while removing sensitivity

Risk Point 2: Vector Database Leakage

- **Embedding Risk:** Model memorization enables reconstruction of proprietary data from embeddings.
- **Insecure Search:** Queries similar to confidential content may retrieve sensitive vectors.
- **Mitigations:**
  - Differential Privacy (DP) during embedding
  - Synthetic data usage
  - Role-based access control on vector stores
  - injection of redundant non-sensitive examples into the vector stores
  - simple duplication (to prevent statistical identification of unique outliers)

Risk Point 3: Retrieved Chunk Leakage

- **Pipeline Risk:** Sensitive chunks retrieved and passed directly to the LLM.
- **Adversarial Potential:** Manipulation of retrieval or reranking to surface confidential content.
- **Balance Point:** Retrieval is the last stage to control exposure before generation.
- **Mitigations:**
  - Indexing and reranking modifications
  - Differential Privacy at cross-attention or reranking stages

Risk Point 4: Answer Leakage

- **LLM Propagation:** Models may introduce sensitive information not present in retrieved chunks.
- **Post-Generation Risk:** Leakage via logs, cached outputs, or conversation histories.
- **Mitigations:**
  - Local LLM deployment
  - Post-generation guardrails and filtering
  - Source attribution and citation enforcement

Risk Point 5: Prompt Leakage

- **User-Side Risk:** Sensitive information in prompts may persist across logs or sessions.
- **Mitigations:**
  - Prompt anonymization and filtering
  - Prompt rewriting and paraphrasing
  - Multi-Party Computation (MPC)

Categorizing Adversarial Manipulation

- **Direct Attacks:**
  - Jailbreaking
  - Prompt Injection
  - Data Poisoning
- **Inference & Extraction Attacks:**
  - Membership Inference
  - Data Extraction from outputs

Dynamic Process View: Setup vs. Inference

- **Pipeline Dependency:** Mitigations at one stage affect all downstream stages.
- **Setup Stage (Indexing):**
  - Document collection and embedding
  - Early anonymization may degrade retrieval quality
- **Inference Stage (Runtime):**
  - Answer filtering and rewriting
  - Better utility, but late-stage privatization

Assessing Mitigation Maturity and Relevance

- **Ideation Gap:** Only 24% of papers discuss deployment costs (latency, overhead).
- **Maturity Trends:**
  - Dataset leakage and poisoning: more mature
  - Prompt injection and extraction: less mature
- **High-Relevance Techniques:**
  - Text filtering and rewriting
  - Differential Privacy (high interest, low maturity)

Measuring Privacy: Evaluation Metrics

- **Retrieval Metrics:**
  - Accuracy (top-k hit rate)
  - Precision, Recall, F1-score
- **Generation Metrics:**
  - ROUGE, BLEU
  - Rejection Rate
- **Attack Metrics:**
  - Attack Success Rate (ASR)
  - Extraction Rate

Technical Trade-offs and Future Research

- **Utility vs. Privacy:** Need for mitigation ensembles and combined impact analysis.
- **Benchmarks:** Lack of RAG-specific privacy benchmarks; reliance on general QA datasets.
- **Architectural Complexity:** Privacy implications of agentic and multi-step RAG systems remain open.

References

Bodea, A.-E., Meisenbacher, S., Klymenko, A., and Matthes, F. (2024).
SoK: Privacy Risks and Mitigations in Retrieval-Augmented Generation Systems.
Technical University of Munich, School of Computation, Information and Technology.

Edge, D., Mody, A., Trinh, H., Truitt, S., Cheng, N., Bradley, J., Metropolitansky, D., Chao, A., Ness, R. O., and Larson, J. (2025).
From Local to Global: A GraphRAG Approach to Query-Focused Summarization.
`arXiv:2404.16130v2 [cs.CL]`, Microsoft Research.

Thank You!